INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	**INTRODUCCIÓN AL DESARROLLO WEB**				
TITULO DE LA PRÁCTICA:	*Fundamentos de JavaScript ECMAScript 6+, Manipulación del DOM y Manejo de Eventos*				
NÚMERO DE PRÁCTICA:	*04*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*2*
FECHA DE PRESENTACIÓN	*15/10/2026*	**HORA DE PRESENTACIÓN**	*21:48*		
INTEGRANTE (s) *Mamani Paucar Francisco Antonio Joan*				**NOTA (0-20)**	*Nota colocada por el docente*
DOCENTE(s): **MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.**					

RESULTADOS Y PRUEBAS

I. EJERCICIOS RESUELTOS:

Demostración de inserción dinámica en el DOM con manejo de eventos y localStorage:
// app.js - Módulo de Gestión const form = document.querySelector('#todo-form'); const input = document.querySelector('#todo-input'); const list = document.querySelector('#todo-list'); let tasks = JSON.parse(localStorage.getItem('tasks')) || []; function renderTasks() { list.innerHTML = ''; tasks.forEach((task, index) => { const li = document.createElement('li'); li.className = 'list-group-item d-flex justify-content-between align-items-center'; li.innerHTML = ` <span>${task.text}</span> <button class="btn btn-danger btn-sm" onclick="deleteTask(${index})">Eliminar</button> `; list.appendChild(li); }); } form.addEventListener('submit', (e) => { e.preventDefault(); const text = input.value.trim(); if (!text) return; tasks.push({ text, completed: false }); localStorage.setItem('tasks', JSON.stringify(tasks)); input.value = ''; renderTasks(); }); function deleteTask(index) { tasks.splice(index, 1); localStorage.setItem('tasks', JSON.stringify(tasks)); renderTasks(); } document.addEventListener('DOMContentLoaded', renderTasks);

II. PROBLEMAS PROPUESTOS RESUELTOS:

Caso Práctico: Aplicación Interactiva de Gestión de Tareas Académicas

Desarrollar una aplicación interactiva "Gestor de Tareas Universitarias" en el navegador cliente sin refresco de página:

- *Gestión del Estado y Estructuras de Datos: Mantener un arreglo de objetos JavaScript global para las tareas: [{ id, titulo, curso, fechaEntrega, completada }].*
- *Utilizar métodos iterativos ES6+ (map, filter, reduce, find) para la manipulación del arreglo.*
-
- *Captura y Validación con JavaScript: Interceptar el evento submit del formulario de alta mediante preventDefault().*

- *Validar mediante JS que ningún campo esté vacío y que la fecha de entrega sea posterior a la fecha actual. En caso de error, mostrar mensajes dinámicos en un div de alertas.*
-
- *Manipulación Dinámica del DOM: Renderizar la lista de tareas dinámicamente construyendo elementos DOM o usando template literals seguros.*
- *Permitir alternar el estado de la tarea (completada/pendiente), modificando sus clases CSS dinámicamente (ej. tachado de texto).*
- *Permitir la eliminación individual de una tarea del DOM y del arreglo.*
- *Implementar un filtro visual (Todas / Pendientes / Completadas) manipulando el DOM.*
-
- *Persistencia de Datos: Sincronizar cada adición, cambio de estado o eliminación con localStorage.*

- Cargar y renderizar las tareas guardadas automáticamente al desencadenarse el evento DOMContentLoaded.

CUERPO HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Gestor de Tareas Universitarias</title>
 <link rel="stylesheet" href="css/style.css">
</head>
<body>
 <main class="app">
  <header class="app__header">
   <h1>Gestor de Tareas Universitarias</h1>
   <p id="resumen" class="resumen" aria-live="polite"></p>
  </header>

  <section class="panel" aria-labelledby="titulo-form">
   <h2 id="titulo-form">Nueva tarea</h2>

   <div id="alertas" class="alertas" role="alert" aria-live="assertive"></div>

   <form id="form-tarea" novalidate>
    <div class="campo">
     <label for="titulo">Título</label>
     <input type="text" id="titulo" name="titulo" placeholder="Ej. Informe de laboratorio
3">
```

```
      </div>
      <div class="campo">
       <label for="curso">Curso</label>
       <input type="text" id="curso" name="curso" placeholder="Ej. Física II">
      </div>
      <div class="campo">
       <label for="fechaEntrega">Fecha de entrega</label>
       <input type="date" id="fechaEntrega" name="fechaEntrega">
      </div>
      <button type="submit" class="btn btn--primario">Agregar tarea</button>
     </form>
    </section>

    <section class="panel" aria-labelledby="titulo-lista">
     <div class="lista-cabecera">
      <h2 id="titulo-lista">Mis tareas</h2>
      <div class="filtros" role="group" aria-label="Filtrar tareas">
       <button type="button" class="filtro activo" data-filtro="todas">Todas</button>
       <button type="button" class="filtro" data-filtro="pendientes">Pendientes</button>
       <button type="button" class="filtro" data-
filtro="completadas">Completadas</button>
      </div>
     </div>

     <ul id="lista-tareas" class="lista"></ul>
    </section>
  </main>

  <script src="app.js"></script>
</body>
</html>

CSS
:root {
  --fondo: #eef2f1;
  --superficie: #ffffff;
  --texto: #1b2a2f;
  --texto-suave: #5b6b70;
  --borde: #cfd9d7;
  --primario: #1f5f5b;
  --primario-osc: #174744;
  --exito: #2e7d4f;
  --error: #b3261e;
```

```
  --error-fondo: #fdecea;
  --vencida: #b3261e;
  --radio: 8px;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  background: var(--fondo);
  color: var(--texto);
  font-family: "Segoe UI", system-ui, -apple-system, sans-serif;
  line-height: 1.5;
}

.app {
  max-width: 720px;
  margin: 0 auto;
  padding: 2rem 1rem 3rem;
}

.app__header h1 {
  margin: 0;
  font-size: 1.9rem;
  letter-spacing: -0.01em;
}

.resumen {
  margin: 0.25rem 0 1.5rem;
  color: var(--texto-suave);
}

.panel {
  background: var(--superficie);
  border: 1px solid var(--borde);
  border-radius: var(--radio);
  padding: 1.25rem;
  margin-bottom: 1.25rem;
}

.panel h2 { margin: 0 0 1rem; font-size: 1.15rem; }

/* Formulario */
```

```
.campo { display: flex; flex-direction: column; margin-bottom: 0.9rem; }
.campo label { font-weight: 600; margin-bottom: 0.25rem; font-size: 0.92rem; }

.campo input {
 padding: 0.6rem 0.7rem;
 border: 1px solid var(--borde);
 border-radius: 6px;
 font: inherit;
 background: #fff;
}

.campo input:focus-visible,
.btn:focus-visible,
.filtro:focus-visible {
 outline: 3px solid #7fb8b3;
 outline-offset: 1px;
}

.campo input.invalido { border-color: var(--error); background: var(--error-fondo); }

.btn {
 font: inherit;
 cursor: pointer;
 border: 0;
 border-radius: 6px;
 padding: 0.6rem 1.1rem;
}

.btn--primario { background: var(--primario); color: #fff; font-weight: 600; }
.btn--primario:hover { background: var(--primario-osc); }

/* Alertas */
.alertas:empty { display: none; }
.alertas {
 background: var(--error-fondo);
 color: var(--error);
 border: 1px solid #f0b8b3;
 border-radius: 6px;
 padding: 0.7rem 1rem;
 margin-bottom: 1rem;
}
.alertas ul { margin: 0; padding-left: 1.1rem; }
.alertas.exito {
```

```
  background: #e8f5ed;
  color: var(--exito);
  border-color: #b5dcc3;
 }

 /* Filtros */
 .lista-cabecera {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: center;
  gap: 0.75rem;
  margin-bottom: 0.5rem;
 }
 .lista-cabecera h2 { margin: 0; }

 .filtros { display: flex; gap: 0.4rem; }

 .filtro {
  font: inherit;
  font-size: 0.9rem;
  cursor: pointer;
  background: transparent;
  border: 1px solid var(--borde);
  border-radius: 999px;
  padding: 0.3rem 0.85rem;
  color: var(--texto-suave);
 }
 .filtro.activo {
  background: var(--primario);
  border-color: var(--primario);
  color: #fff;
 }

 /* Lista */
 .lista { list-style: none; margin: 0; padding: 0; }

 .tarea {
  display: flex;
  align-items: center;
  gap: 0.8rem;
  padding: 0.85rem 0;
  border-bottom: 1px solid var(--borde);
```

```
}
.tarea:last-child { border-bottom: 0; }

.tarea__check { width: 1.2rem; height: 1.2rem; accent-color: var(--primario); cursor:
pointer; }

.tarea__info { flex: 1; min-width: 0; }
.tarea__titulo { font-weight: 600; overflow-wrap: anywhere; }
.tarea__meta { font-size: 0.85rem; color: var(--texto-suave); }
.tarea__meta .vencida { color: var(--vencida); font-weight: 600; }

/* Tarea completada: texto tachado */
.tarea.completada .tarea__titulo {
 text-decoration: line-through;
 color: var(--texto-suave);
}
.tarea.completada .tarea__meta { opacity: 0.7; }

.tarea__eliminar {
 font: inherit;
 cursor: pointer;
 background: transparent;
 color: var(--error);
 border: 1px solid transparent;
 border-radius: 6px;
 padding: 0.3rem 0.6rem;
}
.tarea__eliminar:hover { border-color: var(--error); background: var(--error-fondo); }

.vacio { color: var(--texto-suave); padding: 1rem 0; text-align: center; }

@media (max-width: 480px) {
 .tarea { flex-wrap: wrap; }
 .tarea__eliminar { margin-left: 2rem; }
}

@media (prefers-reduced-motion: reduce) {
 * { transition: none !important; animation: none !important; }
}
```

JavaScript

```
'use strict';
```

```
/* ============================================================
   1. ESTADO GLOBAL
   ============================================================ */
const STORAGE_KEY = 'gestor_tareas_universitarias';

// Arreglo global: [{ id, titulo, curso, fechaEntrega, completada }]
let tareas = [];
let filtroActual = 'todas';

/* Referencias al DOM */
const form = document.getElementById('form-tarea');
const inputTitulo = document.getElementById('titulo');
const inputCurso = document.getElementById('curso');
const inputFecha = document.getElementById('fechaEntrega');
const divAlertas = document.getElementById('alertas');
const listaTareas = document.getElementById('lista-tareas');
const resumen = document.getElementById('resumen');
const botonesFiltro = document.querySelectorAll('.filtro');

/* ============================================================
   4. PERSISTENCIA (localStorage)
   ============================================================ */
function guardarTareas() {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(tareas));
  } catch (e) {
    mostrarAlertas(['No se pudo guardar en el almacenamiento local.']);
  }
}

function cargarTareas() {
  try {
    const datos = JSON.parse(localStorage.getItem(STORAGE_KEY));
    // Se conservan solo los objetos con la estructura esperada
    return Array.isArray(datos)
      ? datos.filter(t => t && t.id && t.titulo && t.curso && t.fechaEntrega)
      : [];
  } catch (e) {
    return [];
  }
}

/* ============================================================
```

```
   Utilidades de fecha
   ========================================================== */
// Convierte 'YYYY-MM-DD' a Date en hora local (evita desfase por zona horaria)
function parsearFecha(texto) {
  const [anio, mes, dia] = texto.split('-').map(Number);
  return new Date(anio, mes - 1, dia);
}

function inicioDeHoy() {
  const ahora = new Date();
  return new Date(ahora.getFullYear(), ahora.getMonth(), ahora.getDate());
}

function formatearFecha(texto) {
  return parsearFecha(texto).toLocaleDateString('es-PE', {
    day: '2-digit', month: 'short', year: 'numeric'
  });
}

/* ==========================================================
   2. CAPTURA Y VALIDACIÓN
   ========================================================== */
function validarFormulario(titulo, curso, fecha) {
  const errores = [];
  const camposInvalidos = [];

  if (!titulo) { errores.push('El título es obligatorio.'); camposInvalidos.push(inputTitulo); }
  if (!curso)  { errores.push('El curso es obligatorio.'); camposInvalidos.push(inputCurso); }

  if (!fecha) {
    errores.push('La fecha de entrega es obligatoria.');
    camposInvalidos.push(inputFecha);
  } else if (parsearFecha(fecha) <= inicioDeHoy()) {
    errores.push('La fecha de entrega debe ser posterior a hoy.');
    camposInvalidos.push(inputFecha);
  }

  return { errores, camposInvalidos };
}

function mostrarAlertas(mensajes, tipo = 'error') {
  divAlertas.className = 'alertas' + (tipo === 'exito' ? ' exito' : '');
  divAlertas.textContent = '';
```

```
  const ul = document.createElement('ul');
  mensajes.forEach(msg => {
   const li = document.createElement('li');
   li.textContent = msg;
   ul.appendChild(li);
  });
  divAlertas.appendChild(ul);
}

function limpiarAlertas() {
  divAlertas.textContent = '';
  divAlertas.className = 'alertas';
  [inputTitulo, inputCurso, inputFecha].forEach(i => i.classList.remove('invalido'));
}

form.addEventListener('submit', (evento) => {
  evento.preventDefault(); // evita el refresco de la página
  limpiarAlertas();

  const titulo = inputTitulo.value.trim();
  const curso = inputCurso.value.trim();
  const fecha = inputFecha.value;

  const { errores, camposInvalidos } = validarFormulario(titulo, curso, fecha);

  if (errores.length > 0) {
   camposInvalidos.forEach(campo => campo.classList.add('invalido'));
   mostrarAlertas(errores);
   camposInvalidos[0].focus();
   return;
  }

  agregarTarea(titulo, curso, fecha);
  form.reset();
  mostrarAlertas(['Tarea agregada correctamente.'], 'exito');
});

/* ============================================================
   Operaciones sobre el arreglo (map, filter, find, reduce)
   ============================================================ */
function generarId() {
  return Date.now().toString(36) + Math.random().toString(36).slice(2, 7);
```

```
}

function agregarTarea(titulo, curso, fechaEntrega) {
 const nueva = { id: generarId(), titulo, curso, fechaEntrega, completada: false };
 tareas = [...tareas, nueva];
 guardarTareas();
 renderizar();
}

function alternarTarea(id) {
 // map: devuelve un nuevo arreglo cambiando solo la tarea elegida
 tareas = tareas.map(t => t.id === id ? { ...t, completada: !t.completada } : t);
 guardarTareas();
 renderizar();
}

function eliminarTarea(id) {
 const tarea = tareas.find(t => t.id === id); // find
 if (!tarea) return;
 if (!confirm(`¿Eliminar la tarea "${tarea.titulo}"?`)) return;

 tareas = tareas.filter(t => t.id !== id); // filter
 guardarTareas();
 renderizar();
}

function obtenerTareasFiltradas() {
 switch (filtroActual) {
  case 'pendientes':  return tareas.filter(t => !t.completada);
  case 'completadas': return tareas.filter(t => t.completada);
  default:        return tareas;
 }
}

function contarPendientes() {
 // reduce: acumula las tareas sin completar
 return tareas.reduce((total, t) => total + (t.completada ? 0 : 1), 0);
}

/* ============================================================
  3. MANIPULACIÓN DINÁMICA DEL DOM
  ============================================================ */
function crearElementoTarea(tarea) {
```

```
  const li = document.createElement('li');
  li.className = 'tarea' + (tarea.completada ? ' completada' : '');
  li.dataset.id = tarea.id;

  const check = document.createElement('input');
  check.type = 'checkbox';
  check.className = 'tarea__check';
  check.checked = tarea.completada;
  check.setAttribute('aria-label', `Marcar "${tarea.titulo}" como completada`);

  const info = document.createElement('div');
  info.className = 'tarea__info';

  const titulo = document.createElement('div');
  titulo.className = 'tarea__titulo';
  titulo.textContent = tarea.titulo; // textContent evita inyección de HTML

  const meta = document.createElement('div');
  meta.className = 'tarea__meta';
  const vencida = !tarea.completada && parsearFecha(tarea.fechaEntrega) <=
inicioDeHoy();
  meta.append(`${tarea.curso} · Entrega: ${formatearFecha(tarea.fechaEntrega)}`);
  if (vencida) {
    const aviso = document.createElement('span');
    aviso.className = 'vencida';
    aviso.textContent = ' (vencida)';
    meta.appendChild(aviso);
  }

  info.append(titulo, meta);

  const btnEliminar = document.createElement('button');
  btnEliminar.type = 'button';
  btnEliminar.className = 'tarea__eliminar';
  btnEliminar.textContent = 'Eliminar';
  btnEliminar.setAttribute('aria-label', `Eliminar "${tarea.titulo}"`);

  li.append(check, info, btnEliminar);
  return li;
}

function renderizar() {
  const visibles = obtenerTareasFiltradas();
```

```
  listaTareas.textContent = '';

  if (visibles.length === 0) {
   const vacio = document.createElement('li');
   vacio.className = 'vacio';
   vacio.textContent = tareas.length === 0
    ? 'Aún no hay tareas. Agrega la primera con el formulario.'
    : 'No hay tareas en esta vista.';
   listaTareas.appendChild(vacio);
  } else {
   const fragmento = document.createDocumentFragment();
   visibles.forEach(t => fragmento.appendChild(crearElementoTarea(t)));
   listaTareas.appendChild(fragmento);
  }

  const pendientes = contarPendientes();
  resumen.textContent = `${tareas.length} en total · ${pendientes} pendiente${pendientes
=== 1 ? '' : 's'}`;

  botonesFiltro.forEach(b => {
   const activo = b.dataset.filtro === filtroActual;
   b.classList.toggle('activo', activo);
   b.setAttribute('aria-pressed', activo);
  });
}

/* Delegación de eventos: un solo listener para toda la lista */
listaTareas.addEventListener('click', (e) => {
  const item = e.target.closest('.tarea');
  if (!item) return;
  const id = item.dataset.id;

  if (e.target.matches('.tarea__eliminar')) eliminarTarea(id);
});

listaTareas.addEventListener('change', (e) => {
  if (!e.target.matches('.tarea__check')) return;
  alternarTarea(e.target.closest('.tarea').dataset.id);
});

/* Filtros */
botonesFiltro.forEach(boton => {
  boton.addEventListener('click', () => {
```

```
    filtroActual = boton.dataset.filtro;
    renderizar();
  });
});

/* Limpia los errores en cuanto la persona corrige el campo */
[inputTitulo, inputCurso, inputFecha].forEach(input => {
  input.addEventListener('input', () => input.classList.remove('invalido'));
});

/* ============================================================
   Inicio: carga automática al terminar de cargar el DOM
   ============================================================ */
document.addEventListener('DOMContentLoaded', () => {
  tareas = cargarTareas();
  renderizar();
});
```

LINK DE GITHUB
https://github.com/f-mamanip/IDWeb/tree/main/Lab04

III. CUESTIONARIO:

1. ¿Cuál es la diferencia de comportamiento entre las palabras clave var, let y const respecto al alcance (scope) y al hoisting?

var posee alcance de función y su hoisting inicializa la variable como undefined, mientras que let y const tienen alcance de bloque, sufren hoisting pero quedan retenidas en la Zona Temporal Muerta (TDZ) hasta su declaración explícita, exigiendo además const un valor inicial inmutable.

2. Explique la técnica de Delegación de Eventos (Event Delegation) en el DOM y por qué es más eficiente que asignar listeners individuales a múltiples elementos.

La delegación de eventos consiste en asignar un único listener a un elemento padre para gestionar las interacciones de sus hijos mediante el burbujeo (bubbling), optimizando el rendimiento al reducir el uso de memoria y soportando automáticamente la incorporación de elementos dinámicos.

3. ¿Por qué es necesario utilizar JSON.stringify() al almacenar objetos complejos en localStorage y qué limitaciones tiene este almacenamiento en términos de seguridad?

JSON.stringify() es indispensable porque localStorage solo almacena cadenas de texto, transformando objetos complejos a formato plano para su guardado; sin embargo, su gran

limitación de seguridad radica en la vulnerabilidad ante ataques XSS, ya que los datos son accesibles globalmente por cualquier script y no cuentan con cifrado nativo.

CONCLUSIONES

El Gestor de Tareas Universitarias demuestra que es posible construir una aplicación interactiva y funcional solo con HTML, CSS y JavaScript, sin librerías externas ni recarga de página. La separación en tres archivos mantiene el código ordenado: HTML define la estructura, CSS el aspecto visual y JavaScript la lógica.

METODOLOGÍA DE TRABAJO

Crear el espacio de trabajo para luego escribir el cuerpo semántico del gestor de tareas.
Luego utilizar CSS para darle un diseño moderno con bordes redondos y fácil de usar.
Luego finalizamos con la implementación de la aplicación en JavaScript.

REFERENCIAS Y BIBLIOGRAFÍA

Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media.
MDN Web Docs. (2025). Document Object Model (DOM) & Events. Developer Mozilla.
Simpson, K. (2015). You Don't Know JS: ES6 & Beyond. O'Reilly Media.